SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science ASSESSMENT

TOOL 1 – Data Interpretation

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 1 – Data Interpretation
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Section / Batch:	AIML	**Date:**	

Code of Conduct

I P. MUNI KARTHIK / 192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3

Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

Annexure A – Data Interpretation

1. Frame Semantics in Banking Customer-Service Systems

A digital banking platform uses semantic frames to understand customer requests. The NLP system identifies the **action, agent, object, and destination** involved in each request.

Customer Query	Generated Semantic Frame
"Transfer ₹20,000 from my savings account."	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)
"Block my debit card immediately."	BLOCK(DebitCard, Customer)
"Send my transaction statement to my email."	SEND(TransactionStatement, Email, Customer)
"Increase my daily withdrawal limit."	MODIFY(WithdrawalLimit, Customer)

Additional transaction logs show:

Query ID	Actual User Intent	System Interpretation
B1	Transfer money to another account	Transfer money
B2	Block debit card	Block debit card
B3	Receive transaction statement by email	Send statement to email
B4	Increase withdrawal limit	Decrease withdrawal limit

Tasks:

1. Analyze the semantic frame of each banking request and identify the relationship between the action and its participants.
2. Identify the query in which the semantic interpretation is incorrect and explain why.
3. Analyze how incorrect identification of semantic roles can lead to an incorrect banking operation.
4. Recommend a frame-based semantic analysis approach to improve the reliability of banking conversational systems.

2. First-Order Predicate Calculus for Smart Agriculture

A smart agricultural system monitors crop fields using sensors and logical rules. **Field Data:**

Field	Soil Condition	Irrigation Status	Crop
F1	Dry	Available	Rice
F2	Wet	Available	Wheat
F3	Dry	Unavailable	Maize
F4	Wet	Unavailable	Rice

The system uses the following predicate rules:

- $Dry(x) \land IrrigationAvailable(x) \rightarrow NeedsWater(x)$
- $NeedsWater(x) \rightarrow Irrigate(x)$
- $Wet(x) \rightarrow \neg NeedsWater(x)$
- $IrrigationUnavailable(x) \rightarrow \neg Irrigate(x)$
- $Rice(x) \land NeedsWater(x) \rightarrow PriorityIrrigation(x)$ **Tasks:**

1. Represent the field observations using First-Order Predicate Calculus.
2. Using the given rules, determine which fields require irrigation.
3. Determine whether F1 and F3 should be treated differently by the automated irrigation controller.
4. Analyze whether predicate logic alone is sufficient for agricultural decision support when sensor information is incomplete or uncertain.

3. Word Sense Disambiguation in a Travel Recommendation System

A travel platform receives search queries containing ambiguous words. The recommendation engine uses surrounding words and user interactions to determine the intended meaning.

Search Query	Possible Sense 1	Possible Sense 2
"Amazon tour"	Amazon rainforest	Amazon company
"Safari booking"	Wildlife tour	Web browser
"Java vacation"	Java island	Java programming language
"Cruise package"	Ship journey	Cruise software/project

The system records the following user interactions:

Query	User Interaction
Amazon tour	Viewed rainforest trekking packages
Safari booking	Selected wildlife resort packages
Java vacation	Viewed hotels in Bali and Java
Cruise package	Viewed Mediterranean ship packages

However, another user searches: **"Safari download for Windows"** and clicks a browser download page.

Tasks:

1. Determine the intended sense of the ambiguous terms using the available context.
2. Identify the lexical and contextual cues that distinguish the possible senses.
3. Analyze why the same word may require different interpretations for different users.
4. Design an industrial-scale WSD strategy using query context, user history, embeddings, and click behaviour to improve travel recommendations.

4. Syntax-Driven Semantic Analysis in Legal Document Processing

A legal NLP system analyzes sentences from contracts and assigns semantic roles based on their syntactic structures.

The system produces the following analyses:

Legal Sentence	Parse Structure	Generated Semantic Interpretation
"The supplier delivered the equipment to the buyer."	Subject–Verb–Object	Supplier = Agent, Equipment = Object, Buyer = Recipient
"The buyer received the equipment from the supplier."	Subject–Verb–Object	Buyer = Agent, Equipment = Object, Supplier = Recipient
"The company terminated the contract after the violation."	Subject–Verb–Object	Company = Agent, Contract = Object
"The contract was terminated by the company."	Passive Construction	Contract = Agent, Company = Object

Tasks:

1. Analyze how syntactic structure, including active and passive constructions, influences semantic-role assignment.
2. Identify the incorrect semantic-role assignments in the given interpretations.
3. Explain how confusing grammatical subject with semantic agent can affect automated legal information extraction.
4. Recommend a syntax-driven semantic analysis architecture that can correctly handle active voice, passive voice, prepositional phrases, and long-distance dependencies in legal documents.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Understanding of Data	Analysis	Application of Concepts	Conclusion	Clarity	Sub. Total	Total %
1	3	3	4	4	4	18	86
2	4	4	4	3	3	18	
3	3	4	3	4	4	18	
4	4	3	4	3	4	18	

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURABAN		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Frame Semantics in Banking Customer-Service Systems

1.1 Semantic frame analysis

Frame semantics represents the meaning of an event by identifying the **action (predicate)** and its associated **participants/semantic roles**.

Customer Query	Semantic Frame	Action	Participants / Roles
"Transfer ₹20,000 from my savings account to my son's account."	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)	Transfer	Customer = Agent, Savings Account = Source, ₹20,000 = Amount, Son's Account = Destination
"Block my debit card immediately."	BLOCK(DebitCard, Customer)	Block	Customer = Agent, Debit Card = Object
"Send my transaction statement to my email."	SEND(TransactionStatement, Email, Customer)	Send	Customer = Agent, Transaction Statement = Object, Email = Destination
"Increase my daily withdrawal limit."	MODIFY(WithdrawalLimit, Customer)	Modify	Customer = Agent, Withdrawal Limit = Object, Increase = Modification

The semantic frame connects the action with the participants involved in that action. This allows the banking system to understand not only what the customer wants but also which account, amount, card, document, or destination is involved.

1.2 Incorrect semantic interpretation The incorrect interpretation is **B4**.

- **Actual intent:** Increase withdrawal limit.
- **System interpretation:** Decrease withdrawal limit.

The system correctly identifies the object as WithdrawalLimit but incorrectly determines the direction of the requested modification.

Therefore:

MODIFY(WithdrawalLimit, Customer, Increase) has

incorrectly been interpreted as:

MODIFY(WithdrawalLimit, Customer, Decrease)

This is a semantic-role/action-value error because the **requested change** has been reversed.

1.3 Effect of incorrect semantic roles

Incorrect semantic-role identification can result in a completely wrong banking operation. For example, if a customer says:

"Increase my daily withdrawal limit to ₹50,000." and the system interprets **increase** as **decrease**, the system might reduce the limit instead of increasing it.

Possible consequences include:

- Customer being unable to withdraw the required amount.
- Incorrect account settings.
- Financial inconvenience to the customer.
- Loss of trust in the banking system.
- Potential financial or regulatory problems if a transaction is processed incorrectly.

In banking, semantic errors are particularly serious because a small misunderstanding can cause an incorrect financial operation.

1.4 Recommended frame-based approach

A reliable banking conversational system should use **frame-based semantic parsing**.

The approach can be:

1. **Intent detection** – identify the customer's main intention such as transfer, block, modify, or statement request.
2. **Entity extraction** – identify accounts, amounts, cards, emails, dates, etc.
3. **Semantic-role labeling** – assign roles such as Agent, Source, Destination, Object and Amount.
4. **Action/modifier detection** – identify words such as *increase*, *decrease*, *cancel*, *block*, and *unblock*.
5. **Frame validation** – verify that all required roles are present and logically consistent.
6. **Confirmation step** – for high-risk operations, confirm the interpreted action with the customer before execution.
7. **Transaction execution** – execute the operation only after successful semantic validation. Thus, frame semantics can significantly improve the accuracy and safety of banking conversational systems.

2. First-Order Predicate Calculus for Smart Agriculture

2.1 Representation of field observations

The observations can be represented using predicates.

F1

- Field(F1)
- Dry(F1)
- IrrigationAvailable(F1)
- Rice(F1)

F2

- Field(F2)
- Wet(F2)
- IrrigationAvailable(F2)
- Wheat(F2)

F3

- Field(F3)
- Dry(F3)
- IrrigationUnavailable(F3)
- Maize(F3)

F4

- Field(F4)
- Wet(F4)
- IrrigationUnavailable(F4)
- Rice(F4)

The general rules can be represented as:

1. $\forall x\ [Dry(x) \land IrrigationAvailable(x) \rightarrow NeedsWater(x)]$
2. $\forall x\ [NeedsWater(x) \rightarrow Irrigate(x)]$
3. $\forall x\ [Wet(x) \rightarrow \neg NeedsWater(x)]$
4. $\forall x\ [IrrigationUnavailable(x) \rightarrow \neg Irrigate(x)]$
5. $\forall x\ [Rice(x) \land NeedsWater(x) \rightarrow PriorityIrrigation(x)]$

2.2 Fields requiring irrigation

F1

F1 is dry and irrigation is available.

Therefore:

$Dry(F1) \land IrrigationAvailable(F1)$

leads to: NeedsWater(F1) Then:

$NeedsWater(F1) \rightarrow Irrigate(F1)$ Therefore,

F1 requires irrigation.

Since F1 contains rice:

$Rice(F1) \land NeedsWater(F1) \rightarrow PriorityIrrigation(F1)$ Therefore,

F1 receives **priority irrigation**.

F2

F2 is wet.

Therefore:

$Wet(F2) \rightarrow \neg NeedsWater(F2)$

Hence, **F2 does not require irrigation**.

F3

F3 is dry, so it requires water:

$Dry(F3) \rightarrow NeedsWater(F3)$ only when irrigation is available. However,

F3 has:

IrrigationUnavailable(F3) Therefore:

$\neg Irrigate(F3)$

Hence, **F3 needs water but cannot currently be irrigated**.

F4

F4 is wet:

$Wet(F4) \rightarrow \neg NeedsWater(F4)$

Therefore, it does not require irrigation. It is also irrigation-unavailable.

Summary

Field	Water Need	Irrigation Availability	Controller Decision
F1	Needs water	Available	**Irrigate**
F2	Does not need water	Available	**Do not irrigate**
F3	Needs water	Unavailable	**Cannot irrigate**
F4	Does not need water	Unavailable	**Do not irrigate**

2.3 Difference between F1 and F3

Yes, **F1 and F3 must be treated differently**.

Both fields are dry, so both indicate a need for water. However:

- F1 → irrigation is available → **irrigate immediately**.
- F3 → irrigation is unavailable → **do not activate irrigation**. For F1, the controller can execute:

Irrigate(F1) For

F3:

¬Irrigate(F3)

The controller could instead generate an alert indicating that F3 requires water but the irrigation system is unavailable.

2.4 Limitations of predicate logic

Predicate logic alone is **not sufficient** for real agricultural decision support.

The rules assume that sensor information is accurate and certain. Real-world sensors can produce:

- Missing readings.
- Incorrect measurements.
- Noisy measurements.
- Conflicting sensor values.
- Uncertain soil moisture values.
- Temporary communication failures.

For example, a sensor might report soil moisture as **49%**, but another sensor may report **53%**. A strict predicate such as Dry(F1) cannot represent this uncertainty effectively.

Therefore, real agricultural systems should combine predicate logic with techniques such as:

- Fuzzy logic.
- Probability-based reasoning.
- Bayesian models.
- Sensor fusion.
- Machine learning.
- Confidence scores.

3. Word Sense Disambiguation in a Travel Recommendation System

3.1 Intended senses

The intended meanings can be determined using the query context and user behaviour.

Query	Intended Sense	Evidence
Amazon tour	Amazon rainforest	User viewed rainforest trekking packages
Safari booking	Wildlife tour	User selected wildlife resort packages
Java vacation	Java island	User viewed hotels in Bali and Java
Cruise package	Ship journey	User viewed Mediterranean ship packages
Safari download for Windows	Web/browser-related software	"Download" and "Windows" strongly indicate software

Thus, the system should not interpret every occurrence of a word using the same meaning.

3.2 Lexical and contextual cues

Different words surrounding an ambiguous term provide useful clues.

Amazon

- **Tour**
- Rainforest
- Trekking
- Wildlife

These strongly suggest **Amazon rainforest**.

In contrast:

- Shopping
- Products
- Prime
- Delivery would suggest **Amazon company**. **Safari**
- Booking
- Wildlife
- Resort
- Animals

suggest **wildlife safari**.

However:

- Download
- Windows
- Browser
- Install

Java

- Vacation
- Hotels
- Bali
- Indonesia suggest **Java island**.

Words such as:

- Programming
- Code
- JVM
- Developer would suggest **Java programming language**. **Cruise**
- Mediterranean
- Ship
- Cabin
- Package suggest **ship journey**.

Terms such as:

- Project
- Software
- Development

could suggest the **Cruise software/project** interpretation.

3.3 Why users require different interpretations The same word can have different meanings depending on:

- Query context.
- Previous searches.
- Click history.
- User interests.
- Location.
- Search session.
- Device.
- Current task.

For example:

User A: “Safari booking”

The likely meaning is a **wildlife safari** because the word *booking* relates naturally to travel.

But:

User B: “Safari download for Windows”

The words *download* and *Windows* strongly indicate the **browser/software sense**.

Therefore, WSD must be **context-dependent and user-dependent**.

3.4 Industrial-scale WSD strategy

A large travel recommendation system can use a multi-stage WSD architecture.

Step 1: Query preprocessing

Clean and tokenize the search query and identify ambiguous words.

Step 2: Context analysis

Analyze nearby words and phrases using NLP techniques.

For example:

Safari + booking + wildlife would produce a strong

travel-related representation.

Step 3: Embedding-based representation

Use contextual language models such as BERT-style transformer models to generate embeddings for:

- Query.
- Candidate senses.
- User history.
- Documents/products.

The system can compare their semantic similarity.

Step 4: User-history analysis Use

previous:

- Searches.
- Clicks.
- Viewed pages.
- Bookings.
- Saved destinations. to determine the user's likely intent.

Step 5: Click behaviour

Clicks provide strong evidence about user intent.

For example:

Safari download for Windows → browser download page provides evidence

that the user means the software rather than wildlife travel.

Step 6: Candidate sense ranking Calculate

a combined score:

Sense Score = Query Context + Embedding Similarity + User History + Click Behaviour

The highest-scoring interpretation becomes the selected sense.

Step 7: Continuous learning

The system should learn from future clicks and bookings so that recommendations improve over time. This combination of **contextual embeddings, user history, and behavioural signals** provides a scalable WSD solution for industrial travel platforms.

4. Syntax-Driven Semantic Analysis in Legal Document Processing

4.1 Influence of syntax on semantic roles

Syntactic structure provides important information for identifying semantic roles, but **grammatical position and semantic role are not always the same**.

Active voice Sentence:

"The supplier delivered the equipment to the buyer." Here:

- Supplier = Agent
- Equipment = Object/Theme
- Buyer = Recipient

The subject **supplier** is also the semantic Agent. **Second sentence**

"The buyer received the equipment from the supplier." Here:

- Buyer = Recipient
- Equipment = Object/Theme
- Supplier = Source/Agent

The original interpretation incorrectly assigns:

Buyer = Agent and

Supplier = Recipient.

The verb **received** changes the semantic roles.

Third sentence

"The company terminated the contract after the violation." Here:

- Company = Agent
- Contract = Object/Theme
- Violation = Cause/Condition

Passive voice

"The contract was terminated by the company." Here:

- Contract = Object/Theme
- Company = Agent

The grammatical subject is **contract**, but it is not the semantic Agent.

Therefore, the generated interpretation:

Contract = Agent, Company = Object

is **incorrect**.

4.2 Incorrect semantic-role assignments There are two major errors.

Error 1

Sentence:

"The buyer received the equipment from the supplier." Incorrect:

- Buyer = Agent
- Supplier = Recipient Correct:
- Buyer = Recipient
- Equipment = Object/Theme
- Supplier = Source/Agent

The word **received** indicates that the buyer obtains something from another participant.

Error 2

Sentence:

"The contract was terminated by the company." Incorrect:

- Contract = Agent
- Company = Object Correct:
- Contract = Object/Theme
- Company = Agent

The passive construction places the semantic object in the grammatical subject position.

4.3 Problems caused by confusing subject and agent A legal NLP system that assumes: **Subject = Agent** can produce serious errors.

For example:

"The contract was terminated by the company."

If the system identifies the contract as the Agent, it may incorrectly conclude that the **contract performed the termination**.

This can affect:

- Contract analysis.
- Liability detection.
- Responsibility identification.
- Legal event extraction.
- Compliance analysis.

4.4 Recommended syntax-driven architecture

A robust legal semantic analysis system should use the following pipeline:

Legal Document → Sentence Segmentation → Tokenization → POS Tagging → Dependency Parsing → Semantic Role Labeling → Coreference Resolution → Semantic Representation → Legal Information Extraction

1. Legal preprocessing Perform:

- Sentence segmentation.
- Tokenization.
- Part-of-speech tagging.
- Legal terminology recognition.

2. Dependency parsing

Construct relationships between words.

For example:

company → terminated → contract and

identify passive structures such as:

contract ← terminated ← company

3. Semantic Role Labeling Assign roles such as:

- Agent.
- Patient/Theme.
- Recipient.
- Source.
- Instrument.
- Cause.
- Time.

4. Active/passive detection

The system must explicitly detect voice.

Active:

Company → terminated → Contract **Passive:**

Contract → was terminated → by Company The semantic roles remain:

Company = Agent Contract = Theme even though their grammatical positions change.

5. Prepositional phrase analysis

Prepositions such as **by, from, to, after, before, with** provide important semantic information.

For example:

- **by the company** → Agent.
- **to the buyer** → Recipient.
- **from the supplier** → Source.
- **after the violation** → Temporal relation.

6. Long-distance dependency handling

Legal documents frequently contain long and complex sentences. Transformer-based models and dependency graphs can help connect entities with predicates even when several words or clauses occur between them.

7. Coreference resolution

The system should resolve references such as:

"The supplier delivered the equipment. **It** was inspected by the buyer." The system must determine what **"it"** refers to.

8. Legal validation layer

Finally, extracted semantic relationships should be checked against legal rules and document context before being used for automated decision-making.

Therefore, a **syntax-driven semantic-role architecture combined with dependency parsing, semanticrole labeling, voice detection, coreference resolution, and contextual transformer models** would provide much more reliable legal information extraction.